

Le RAG de Ghabetna

Enchaînement des processus et explication ligne par ligne de `rag/chunker.py`

`backend/chatbot_ms`

Contents

1	Vue d'ensemble : les deux processus du RAG	2
1.1	Processus A : indexation des données	2
1.2	Processus B : traitement d'une question utilisateur	3
2	Explication ligne par ligne de <code>rag/chunker.py</code>	5
2.1	En-tête, docstring et imports (lignes 1 à 30)	5
2.2	Les expressions régulières (lignes 31 à 54)	7
2.3	Utilitaires (lignes 55 à 97)	10
2.4	<code>decouper_sections</code> : lire la structure du document (lignes 98 à 134)	12
2.5	<code>separer_contenu</code> : trier normatif, avertissements et notes (lignes 135 à 187)	14
2.6	Tableaux et articles trop longs (lignes 188 à 269)	17
2.7	Préfixes contextuels (lignes 270 à 327)	20
2.8	<code>charger_manifeste</code> (lignes 328 à 345)	23
2.9	<code>chunker_fichier</code> : le cœur du découpage (lignes 346 à 438)	24
2.10	<code>_fabriquer</code> : la forme d'un chunk (lignes 439 à 479)	29
2.11	<code>chunker_corpus</code> : le point d'entrée (lignes 480 à 523)	31
2.12	<code>_rapport</code> et exécution directe (lignes 524 à 551)	33
3	Remarques relevées pendant la lecture	35
4	Résumé : le rôle de <code>chunker.py</code> dans le RAG	35

1 Vue d'ensemble : les deux processus du RAG

Le RAG de Ghabetna n'est pas une seule chaîne de fichiers exécutés l'un après l'autre. Il est composé de **deux processus distincts** qui ne s'exécutent pas au même moment et ne passent pas par les mêmes fichiers :

1. **L'indexation** (hors ligne) : exécutée une seule fois, puis à chaque modification du corpus. Elle transforme les fichiers `.md` en index consultables.
2. **La réponse à une question** (en ligne) : exécutée à chaque requête d'un utilisateur. Elle lit les index déjà construits.

Trois précisions importantes sur l'ordre des fichiers :

- `store.py` n'intervient **pas** pendant l'indexation : il ne construit rien, il *lit* les fichiers produits par l'indexation, au moment de répondre.
- `synonymes.py` et `calendrier.py` n'interviennent **pas** pendant l'indexation : ils travaillent uniquement sur la question de l'utilisateur.
- Le cache est **consulté en premier**, avant toute recherche, et **alimenté en dernier**, après la génération.

`config.py` est utilisé par tous les fichiers : il regroupe tous les réglages (chemins, modèle, tailles, clés).

1.1 Processus A : indexation des données

Commande : `python -m scripts.build_index`

```
scripts/build_index.py          (chef d'orchestre : lance 2 processus Python separees)
|
+-- PROCESSUS 1 : scripts/build_embeddings.py          (lourd : charge torch)
|
|   +-- ETAPE 1 : rag/chunker.py -> chunker_corpus()
|   |   lit    data/corpus/<domaine>/*.md + _manifest.json
|   |   ecrit  data/index/chunks.json          (~320 chunks)
|   |
|   +-- ETAPE 2 : rag/indexer.py -> embedder(chunks)
|   |   charge le modele bge-m3, transforme chaque chunk en vecteur
|   |   ecrit  data/index/vecteurs.npy          (matrice 320 x 1024)
|   |   data/index/vecteurs.meta.json          (empreinte du cache)
|   |
+-- PROCESSUS 2 : scripts/build_stores.py          (leger : sans torch)
|
|   +-- ETAPE 3 : rag/indexer.py -> indexer_bm25(chunks)
|   |   ecrit  data/index/bm25.pkl              (index lexical)
```

Pourquoi deux processus ? L'embedding est l'étape lente (environ 11 minutes sur CPU). Comme son résultat, `vecteurs.npy`, est enregistré sur disque, l'étape BM25 peut être relancée en quelques secondes sans recharger le modèle. Séparer les processus évite aussi les conflits de bibliothèques natives (DLL) entre `torch` et les autres bibliothèques.

Résultat : trois fichiers dans `data/index/`.

Fichier	Contenu
<code>chunks.json</code>	la liste des chunks : textes et métadonnées
<code>vecteurs.npy</code>	un vecteur de 1024 nombres par chunk, dans le même ordre ; c'est le « magasin vectoriel » lu par <code>store.py</code>
<code>bm25.pkl</code>	l'index lexical BM25 et la liste des identifiants dans le même ordre

1.2 Processus B : traitement d'une question utilisateur

Au démarrage du service (fonction `lifespan` de `app/main.py`), une seule fois :

```
Retriever() -> charge le modele bge-m3 (10 a 20 s)
             -> NumpyStore() lit vecteurs.npy + chunks.json      (store.py)
             -> lit bm25.pkl
```

Puis à chaque appel de POST `/api/chat/` :

```
Flutter envoie { question, historique, specialite }
|
(1) config.domaines_pour("chasseur")          -> ["chasse", "app"]
|
(2) calendrier.detecter_date(question)         -> la question contient-elle une date ?
|
(3) cache.lire()      (seulement si : pas d'historique ET pas de date)
    |-- trouve -> reponse immediate, FIN
    |
(4) retriever.rechercher(question, domaines, question_precedente)
    |   +--- _dense() : bge-m3 encode la question -> store.query()      (le SENS)
    |   +--- _bm25()  : synonymes.enrichir(question) -> tokeniser -> BM25  (les MOTS)
    |   +--- _rrf()   : fusion des deux classements -> top 8 chunks
    |
(5) calendrier.analyser_question_struct(question)
    |   espece + date ? -> verdict calcule en Python (autorise / refuse / passee)
    |
(6) generator.generer(question, chunks, historique, specialite, verdict)
    |   prompt systeme + [VERDICT] + [EXTRAITS] + QUESTION -> Groq (LLM)
    |
(7) cache.ecrire()    (memes conditions qu'en (3))
    |
(8) reponse JSON : { reponse, verdict, sources, domaines, depuis_cache, duree_ms }
```

Où interviennent synonymes et calendrier ?

- `synonymes.py` est appelé à l'intérieur du `retriever`, et **uniquement** du côté BM25 : on ajoute à la question les mots du vocabulaire officiel (« prix » → « redevance, montant, dinars »). On ne le fait pas côté dense, car ajouter des mots déplacerait le vecteur de la question vers une moyenne floue.

- `calendrier.py` est appelé à *côté* du `retrieve`, pas dedans. Il sert deux fois : à l'étape (2) pour décider si on utilise le cache, et à l'étape (5) pour calculer le verdict.

Pourquoi le cache est-il désactivé avec un historique ou une date ? Avec un historique, la même question peut appeler une réponse différente selon ce qui précède (« et pour le lièvre ? »). Avec une date relative, « demain » ne désigne pas le même jour d'un jour à l'autre. Dans ces deux cas, une réponse en cache serait fausse.

2 Explication ligne par ligne de rag/chunker.py

Rôle du fichier

chunker.py est la **première étape** de l'indexation. Il lit les fichiers Markdown de `data/corpus/<domaine>/` et les découpe en petits morceaux autonomes appelés *chunks*, enregistrés dans `data/index/chunks.json`.

Sa règle fondamentale est : **un chunk = un article de loi**. Le découpage est *sémantique* : on cherche le motif « Article N » dans les titres, au lieu de se fier au niveau `##` ou `###`, qui varie d'un fichier à l'autre.

Exemple de chunk réellement produit

Extrait de `chunks.json` (première ligne du tableau de l'article premier de l'arrêté) :

```
{
  "id": "arrete_2025_art1_lievre_perdrix_caille_sedentai",
  "texte_embedding": "Arrêté chasse 2025/2026, Article premier. Lièvre, perdrix, ...
                     Lièvre, perdrix, ... Date d'ouverture : 5 octobre 2025, ...",
  "texte_index": "Arrêté du 27 août 2025 (JORT n° 107 du 29 août 2025) - Titre
                 premier : Réglementation générale - Article premier - Lièvre, ...",
  "texte_affiche": "Lièvre, perdrix, ... Date d'ouverture : 5 octobre 2025,
                   Date de fermeture : 7 décembre 2025.",
  "metadata": { "domaine": "chasse", "source_id": "arrete_2025", "article": "1",
                "type": "tableau", "validite": "saison_2025_2026", ... }
}
```

Comment lire la suite

Le code est reproduit **intégralement**, par blocs, avec les numéros de ligne réels du fichier. Sous chaque bloc, chaque ligne est expliquée ([L.n](#)). Les lignes vides et les commentaires sont aussi signalés, pour qu'aucune ligne ne soit sautée.

2.1 En-tête, docstring et imports (lignes 1 à 30)

```
1 #!/usr/bin/env python3
2 # -*- coding: utf-8 -*-
```

L. 1 *Shebang*. Sous Linux ou macOS, si on exécute le fichier directement (`./chunker.py`), le système lit cette ligne pour savoir quel interpréteur utiliser. `/usr/bin/env python3` cherche `python3` dans le `PATH`. Cette ligne est sans effet sous Windows et quand on lance le module avec `python -m`.

L. 2 Déclaration d'encodage (norme PEP 263). En Python 3, l'UTF-8 est déjà l'encodage par défaut : la ligne est gardée par convention. Elle rappelle que le fichier contient des caractères accentués (é, «, »).

```

3 """
4 rag/chunker.py - ETAPE 1 : decoupage des .md en chunks
5 =====
6
7 REGLE FONDAMENTALE :
8     Un chunk = un ARTICLE de loi.
9     Le decoupage est SEMANTIQUE (on cherche le motif "Article N" dans le
10    titre) et non SYNTAXIQUE : on ne peut pas se baser sur le niveau ## ou
11    ###, car il varie d'un fichier a l'autre. Dans l'arrete les articles
12    sont en H2 ; dans le code forestier ils sont en H2 pour le Titre I et
13    en H3 pour le Titre II.
14
15 MULTI-DOMAINES :
16     Le nom du sous-dossier de data/corpus/ DEVIENT la metadonnee "domaine".
17     Ajouter l'apiculture = creer data/corpus/apiculture/ et y depoter des .md.
18     Aucune ligne de code a modifier.
19
20 Ce module ne depend d'aucune librairie externe : il se teste seul.
21 """

```

L. 3 Ouverture d'une chaîne entre triples guillemets. Placée en tout début de module, elle devient la *docstring* du module : ce n'est pas du code exécuté mais une documentation, accessible par `chunker.__doc__` ou `help(chunker)`.

L. 4–5 Titre du module et soulignement décoratif.

L. 6 Ligne vide de mise en forme.

L. 7–13 La règle fondamentale : un chunk = un article. On repère les articles par le motif « Article N » et non par le niveau de titre, car dans l'arrêté les articles sont en ## (H2), alors que dans le code forestier ils sont en H2 pour le Titre I et en H3 pour le Titre II.

L. 14 Ligne vide.

L. 15–18 Principe multi-domaines : le nom du sous-dossier de `data/corpus/` devient la métadonnée `domaine`. Ajouter un domaine consiste à créer un dossier et à y déposer des `.md`, sans modifier le code.

L. 19 Ligne vide.

L. 20 Le module n'utilise que la bibliothèque standard : on peut le tester sans installer `torch` ni `sentence-transformers`.

L. 21 Fermeture de la docstring.

```

22
23 from __future__ import annotations
24
25 import json
26 import re
27 import unicodedata
28 from pathlib import Path
29
30 from . import config

```

L. 22 Ligne vide.

L. 23 `from __future__ import annotations` (PEP 563) : les annotations de type ne sont plus évaluées à l'exécution mais conservées sous forme de texte. Cela permet d'écrire `str | None` ou `dict[int, str]` même sur une version de Python antérieure à 3.10. Cet import doit être la première instruction du fichier, juste après la docstring.

L. 24 Ligne vide.

L. 25 `json` : lire les `_manifest.json` et écrire `chunks.json`.

L. 26 `re` : expressions régulières, pour reconnaître titres, articles et tableaux.

L. 27 `unicodedata` : base de données Unicode, utilisée pour retirer les accents.

L. 28 `Path` : manipulation de chemins de fichiers orientée objet (module `pathlib`).

L. 29 Ligne vide.

L. 30 `from . import config : import relatif`. Le point désigne le paquet courant, `rag`. On récupère ainsi `config.TAILLE_MAX_CHUNK`, `config.CORPUS_DIR`, etc. Conséquence : le fichier doit être lancé comme module (`python -m rag.chunker`) ; un lancement direct (`python rag/chunker.py`) lève *attempted relative import with no known parent package*.

2.2 Les expressions régulières (lignes 31 à 54)

```
31
32 # =====
33 # EXPRESSIONS REGULIERES
34 # =====
35
36 RE_TITRE = re.compile(r"^(#{1,6})\s+(.+?)\s*##\$")
```

Toutes les expressions régulières sont *compilées* une seule fois, au chargement du module, avec `re.compile`. Elles sont ensuite réutilisées sans être réanalysées. Le préfixe `r"..."` (*raw string*) empêche Python d'interpréter les `\` : ainsi `\s` arrive tel quel au moteur d'expressions régulières.

L. 31 Ligne vide.

L. 32–34 Bannière de commentaire qui délimite la section.

L. 35 Ligne vide.

L. 36 `RE_TITRE` reconnaît un titre Markdown. Décomposition :

- `^` : début de ligne ;
- `(#{1,6})` : **groupe 1**, de 1 à 6 dièses ; son nombre de caractères donnera le niveau du titre ;
- `\s+` : au moins un espace, donc `#hashtag` n'est pas un titre ;
- `(.+?)` : **groupe 2**, le texte du titre ; le `?` rend le quantificateur *paresseux* (il prend le moins de caractères possible) ;
- `\s*##` : espaces et dièses fermants facultatifs (syntaxe `## Titre ##`) ; grâce au quantificateur paresseux, ils ne sont pas avalés par le groupe 2 ;
- `$` : fin de ligne.

Exemple : `### Article 165` donne groupe 1 = `###` et groupe 2 = `Article 165`.

```

37
38 # Detecte un ARTICLE quel que soit son niveau de titre.
39 # Gere "Article premier", "Article 12", "Article 134 bis".
40 RE_ARTICLE = re.compile(r"^Article\s+(premier|\d+\s*bis|\d+)\b", re.IGNORECASE)

```

L. 37 Ligne vide.

L. 38–39 Commentaires : la regex détecte un article quel que soit son niveau de titre.

L. 40 RE_ARTICLE reconnaît un titre d'article :

- `^Article\s+` : le mot « Article » en début de chaîne, suivi d'espaces ;
- `(premier|\d+\s*bis|\d+)` : **groupe 1**, trois alternatives. **L'ordre compte** : le moteur essaie les alternatives de gauche à droite. Si `\d+` était placé avant `\d+\s*bis`, « 134 bis » serait capturé comme « 134 ». `\s*` accepte « 134 bis » comme « 134bis » ;
- `\b` : *frontière de mot*. Elle empêche une correspondance partielle (« premiers ») ;
- `re.IGNORECASE` : insensible à la casse (« ARTICLE 5 », « article 5 »).

Un titre comme « Article 75 (Modifié par la loi n° 2005-13) » est reconnu, car seul le début est testé.

Attention : « Article 1er » n'est pas reconnu : entre « 1 » et « er » il n'y a pas de frontière de mot, donc `\b` échoue. Le corpus actuel écrit « Article premier ».

```

41
42 # Sous-section géographique de l'article 12 de l'arrete
43 RE_GOUVERNORAT = re.compile(r"^Gouvernorat\s+(?:de\s+l'|\du\s+|\de\s+|\d')?(.+)$", re.I)

```

L. 41 Ligne vide.

L. 42 Commentaire : l'article 12 de l'arrêté est divisé en sous-sections par gouvernorat.

L. 43 RE_GOUVERNORAT :

- `^Gouvernorat\s+` : le mot « Gouvernorat » suivi d'espaces ;
- `(?:de\s+l'|\du\s+|\de\s+|\d')?` : groupe *non capturant* `((?:...))`, facultatif `(?)`, qui absorbe l'article : « de l' », « du », « de », « d' ». Ici aussi l'ordre compte : `de\s+l'` doit passer avant `de\s+`, sinon « de l'Ariana » donnerait « l'Ariana » ;
- `(.+)$` : **groupe 1**, le nom du gouvernorat jusqu'à la fin ;
- `re.I` : abréviation de `re.IGNORECASE`.

Exemples : « Gouvernorat de l'Ariana » → « Ariana » ; « Gouvernorat du Kef » → « Kef » ; « Gouvernorat de Béja » → « Béja ».

```

44
45 # Les blocs > ajoutés dans les .md : deux types traités différemment
46 RE_NOTE_FIDELITE = re.compile(r"\.*Note de fidélité\.*\.*Périmètre\.*\.*", re.I)
47 RE_AVERTISSEMENT = re.compile(r"\.*Avertissement de concordance\.*\.*", re.I)

```


2.3 Utilitaires (lignes 55 à 97)

```
55
56
57 # =====
58 # UTILITAIRES
59 # =====
60
61 def plier_accents(txt: str) -> str:
62     """
63     Aïn -> Ain, Béjà -> Beja.
64     Sert UNIQUEMENT a fabriquer des identifiants techniques.
65     Le texte affiche garde ses accents.
66     """
67     d = unicodedata.normalize("NFD", txt)
68     return "".join(c for c in d if unicodedata.category(c) != "Mn")
```

L. 55–56 Deux lignes vides entre sections de module (convention PEP 8).

L. 57–59 Bannière « UTILITAIRES ».

L. 60 Ligne vide.

L. 61 Définition de `plier_accents`. `txt: str` et `-> str` sont des annotations de type : documentation pour le lecteur et l'éditeur, non vérifiées à l'exécution.

L. 62–66 Docstring : la fonction transforme « Aïn » en « Ain ». Elle sert *uniquement* à fabriquer des identifiants ; le texte affiché garde ses accents.

L. 67 `unicodedata.normalize("NFD", txt)` : forme de normalisation *NFD* (décomposition canonique). Chaque caractère accentué est séparé en lettre de base et accent *combinant* : « é » (U+00E9) devient « e » (U+0065) suivi de « ´ » (U+0301).

L. 68 Expression génératrice : on parcourt chaque caractère `c` et on ne garde que ceux dont la catégorie Unicode n'est pas "Mn" (*Mark, nonspacing*, c'est-à-dire les accents combinants). `"".join(...)` recolle les caractères restants. Résultat : « Béja » → « Beja ». Les ligatures comme « œ » ne sont pas décomposées.

```
69
70
71 def slug(txt: str, taille: int = 40) -> str:
72     """'Sidi Bouzid' -> 'sidi_bouzid'"""
73     t = plier_accents(txt.lower())
74     return re.sub(r"[^a-z0-9]+", "_", t).strip("_")[:taille]
```

L. 69–70 Lignes vides.

L. 71 `slug(txt, taille=40)` : `taille` est un paramètre avec *valeur par défaut* ; on peut l'omettre à l'appel.

L. 72 Docstring sur une ligne : exemple d'entrée et de sortie.

L. 73 Passage en minuscules, puis suppression des accents.

L. 74 Trois opérations enchaînées :

- `re.sub(r"[^a-z0-9]+", "_", t)` remplace toute suite de caractères qui ne sont ni lettres ni chiffres (classe négative, + = une ou plusieurs) par un seul _ ;

- `.strip("_")` retire les `_` au début et à la fin ;
- `[:taille]` : *slicing*, on garde au plus `taille` caractères.

Exemple : « Gouvernorat de l'Ariana » → `gouvernorat_de_l_ariana`.

```

75
76
77 def numero_article(titre: str) -> str | None:
78     """
79     'Article premier' -> '1' | 'Article 134 bis' -> '134bis'
80     None si ce n'est pas un article (Chapitre, Visas, Preamble...).
81     """
82     m = RE_ARTICLE.match(titre)
83     if not m:
84         return None
85     num = m.group(1).lower().replace(" ", "")
86     return "1" if num == "premier" else num

```

L. 75–76 Lignes vides.

L. 77 `-> str | None` : la fonction renvoie soit une chaîne, soit `None`.

L. 78–81 Docstring : « Article premier » → "1", « Article 134 bis » → "134bis", `None` pour un titre qui n'est pas un article.

L. 82 `RE_ARTICLE.match(titre)` : `match` ne teste que le *début* de la chaîne (contrairement à `search`, qui cherche partout). Renvoie un objet `Match` ou `None`.

L. 83 `if not m` : `None` est « faux » en Python.

L. 84 Pas un article (« Chapitre II », « Visas »...) : on renvoie `None`.

L. 85 `m.group(1)` est le texte capturé par le groupe 1. `.lower()` normalise la casse (grâce à `IGNORECASE`, on a pu capturer « Premier » ou « BIS »). `.replace(" ", "")` transforme « 134 bis » en « 134bis ».

L. 86 *Expression conditionnelle* (`A if condition else B`) : « premier » devient "1", sinon on renvoie le numéro tel quel.

```

87
88
89 def extraire_alias_sic(texte: str) -> list[str]:
90     """
91     Recupere les formes correctes derriere les *(sic : « ... »)*.
92     L'article 176 ecrit "chausseurs" : coquille du texte OFFICIEL.
93     On garde "chausseurs" a l'affichage (fidelite juridique) mais on ajoute
94     "chasseurs" comme alias pour que BM25 puisse le retrouver.
95     """
96     return [m.group(1).strip() for m in RE_SIC.finditer(texte)
97             if m.group(1).strip() and len(m.group(1).strip()) < 60]

```

L. 87–88 Lignes vides.

L. 89 Définition : reçoit un texte, renvoie une liste de chaînes.

L. 90–95 Docstring : l'article 176 écrit « chasseurs », une coquille du texte *officiel*. On la garde à l'affichage pour rester fidèle au texte juridique, mais on ajoute « chasseurs » comme alias pour que BM25 retrouve l'article.

L. 96–97 *Compréhension de liste* :

- `RE_SIC.finditer(texte)` parcourt *toutes* les occurrences, une par une (contrairement à `match`, qui ne teste que le début) ;
- `m.group(1).strip()` : la forme correcte, sans espaces autour ;
- filtre : on garde l'alias s'il n'est pas vide et fait moins de 60 caractères, ce qui écarte les captures aberrantes trop longues.

L'expression `m.group(1).strip()` est calculée trois fois par occurrence : c'est un peu redondant, mais le coût est négligeable.

2.4 découper_sections : lire la structure du document (lignes 98 à 134)

```
98
99
100 # =====
101 # DECOUPAGE EN SECTIONS
102 # =====
103
104 def decouper_sections(texte: str) -> list[dict]:
105     """
106     Produit une liste de sections, chacune avec LA PILE DE SES PARENTS.
107
108     La pile est essentielle : "### Article 165" ne dit pas à quel Titre ni à
109     quel Chapitre il appartient. Ce sont ses parents qui le disent, et c'est
110     cette information qui constituera le préfixe contextuel.
111     """
112     sections, courante = [], None
113     parents: dict[int, str] = {}
```

C'est la fonction clé du fichier. Elle lit le Markdown ligne par ligne et produit une liste de sections. Chaque section mémorise **la pile de ses titres parents**.

L. 98–99 Lignes vides.

L. 100–102 Bannière « DECOUPAGE EN SECTIONS ».

L. 103 Ligne vide.

L. 104 Définition : reçoit le texte complet d'un fichier et renvoie une liste de dictionnaires.

L. 105–111 Docstring : « `### Article 165` » ne dit pas à quel Titre ni à quel Chapitre il appartient. Ce sont ses parents qui le disent, et ils serviront au préfixe contextuel.

L. 112 Affectation multiple (*déballage de tuple*) : `sections = []` contiendra le résultat ; `courante = None` est la section en cours de remplissage (aucune au départ).

L. 113 `parents` est un dictionnaire `{niveau : titre}`, par exemple `{1: "Titre II", 2: "Chapitre I"}`. L'annotation `dict[int, str]` documente les types.

```

114
115     for ligne in texte.splitlines():
116         m = RE_TITRE.match(ligne)
117         if m:
118             niveau, titre = len(m.group(1)), m.group(2).strip()
119             if courante:
120                 sections.append(courante)
121                 # On oublie les parents de niveau >= au notre : en arrivant sur
122                 # "## Chapitre II" on oublie les "###" du chapitre precedent,
123                 # mais on garde le "# Titre II".
124                 parents = {k: v for k, v in parents.items() if k < niveau}
125                 courante = {"niveau": niveau, "titre": titre,
126                             "parents": [parents[k] for k in sorted(parents)],
127                             "lignes": []}
128                 parents[niveau] = titre
129             elif courante is not None:
130                 courante["lignes"].append(ligne)

```

L. 114 Ligne vide.

L. 115 `texte.splitlines()` découpe le texte en lignes. Contrairement à `split("\n")`, elle gère aussi les fins de ligne Windows `\r\n`.

L. 116 On teste si la ligne est un titre Markdown.

L. 117 `if m:` : un objet `Match` est « vrai », `None` est « faux ».

L. 118 `len(m.group(1))` compte les dièses : `###` donne le niveau 3. `m.group(2).strip()` donne le texte du titre sans espaces.

L. 119–120 Un nouveau titre commence : la section précédente est terminée et on l'ajoute au résultat. Au tout premier titre, `courante` vaut `None` et rien n'est ajouté.

L. 121–123 Commentaire qui explique la ligne suivante.

L. 124 *Compréhension de dictionnaire* : on ne garde que les parents de niveau **strictement inférieur** au titre courant. En arrivant sur « `## Chapitre II` », on oublie les « `###` » du chapitre précédent, mais on garde « `# Titre II` ».

L. 125–127 Création de la nouvelle section :

- "niveau" et "titre" : ceux du titre lu ;
- "parents" : `sorted(parents)` trie les *clés* (les niveaux), puis on prend les titres dans cet ordre : du plus général au plus précis ;
- "lignes" : liste vide, remplie par les lignes suivantes.

La pile est calculée *avant* la ligne 128 : une section n'est donc jamais son propre parent.

L. 128 Le titre courant devient parent des sections qui suivront.

L. 129 `elif courante is not None` : ce n'est pas un titre et une section est ouverte.

L. 130 La ligne brute (non nettoyée) est ajoutée au contenu de la section courante.

Attention : les lignes situées **avant le premier titre** du fichier ne vont nulle part : `courante` vaut encore `None`, elles sont perdues sans avertissement.

```

131
132     if courante:
133         sections.append(courante)
134     return sections

```

L. 131 Ligne vide.

L. 132–133 À la fin de la boucle, la dernière section n’a pas été ajoutée, car aucun titre suivant n’a déclenché la ligne 120. On l’ajoute ici.

L. 134 Renvoi de la liste des sections.

Exemple d’évolution de la pile :

Ligne lue	parents après L.124	Pile enregistrée
# Titre II	{}	[]
## Chapitre I	{1: Titre II}	[Titre II]
### Article 165	{1: Titre II, 2: Chap. I}	[Titre II, Chapitre I]
### Article 166	{1: Titre II, 2: Chap. I}	[Titre II, Chapitre I]
## Chapitre II	{1: Titre II}	[Titre II]

2.5 `separer_contenu` : trier normatif, avertissements et notes (lignes 135 à 187)

```

135
136
137 # =====
138 # SEPARATION NORMATIF / NOTES
139 # =====
140
141 def separer_contenu(lignes: list[str]) -> tuple[str, list[str], list[str]]:
142     """
143     Trie le contenu en trois categories :
144
145     1. normatif          -> le texte de loi lui-meme
146     2. avertissements -> blocs "> **Avertissement de concordance**"
147                        A INDEXER : ils empechent une reponse fausse
148                        (renvoi perime de la loi 69-33 vers l'article 161).
149     3. notes             -> blocs "> **Note de fidélité**", purement editoriaux,
150                        exclus de l'index.
151     """
152     normatif, avertissements, notes = [], [], []
153     bloc, type_bloc = [], None

```

L. 135–136 Lignes vides.

L. 137–139 Bannière « SEPARATION NORMATIF / NOTES ».

L. 140 Ligne vide.

L. 141 Définition : reçoit les lignes d’une section et renvoie un *tuple* de trois éléments : le texte normatif (`str`), la liste des avertissements et la liste des notes.

L. 142–151 Docstring qui décrit les trois catégories :

- **normatif** : le texte de loi lui-même ;

- **avertissements** : blocs « Avertissement de concordance ». À *indexer* : ils empêchent une réponse fausse (le renvoi périmé de la loi 69-33 vers l'article 161) ;
- **notes** : blocs « Note de fidélité », purement éditoriaux, exclus de l'index.

L. 152 Trois listes vides.

L. 153 `bloc` accumule les lignes du bloc `>` en cours ; `type_bloc` vaut `None` (aucun bloc), `"avertissement"` ou `"note"`.

```

154
155     def cloturer():
156         nonlocal bloc, type_bloc
157         if bloc:
158             contenu = "\n".join(bloc).strip()
159             (avertissements if type_bloc == "avertissement" else notes).append(contenu)
160             bloc, type_bloc = [], None

```

L. 154 Ligne vide.

L. 155 `cloturer` est une *fonction imbriquée* (définie dans une autre fonction). Elle a accès aux variables de `separer_contenu` : c'est une *fermeture* (closure). Elle termine le bloc `>` en cours.

L. 156 `nonlocal bloc, type_bloc` est indispensable, car la ligne 160 *réaffecte* ces variables. Sans `nonlocal`, Python les considérerait comme locales à `cloturer`, et la ligne 157 lèverait `UnboundLocalError`. Les listes `avertissements` et `notes` n'en ont pas besoin : elles sont seulement modifiées (`.append`), pas réaffectées.

L. 157 Un bloc est-il ouvert ? Une liste non vide est « vraie ».

L. 158 On recolle les lignes du bloc avec des retours à la ligne, puis on retire les espaces aux extrémités.

L. 159 L'expression conditionnelle choisit *la liste de destination*, puis on appelle `.append` dessus : `avertissement` si le type est `"avertissement"`, `note` sinon.

L. 160 Réinitialisation : plus de bloc en cours.

```

161
162     for ligne in lignes:
163         nue = ligne.strip()
164
165         if nue.startswith(">"):
166             contenu = nue.lstrip("> ").rstrip()
167             if type_bloc is None:
168                 type_bloc = ("avertissement" if RE_AVERTISSEMENT.search(contenu)
169                             else "note")
170             bloc.append(contenu)
171             continue

```

L. 161 Ligne vide.

L. 162 Parcours de chaque ligne de la section.

L. 163 `nue` est la ligne sans espaces ni `\r` aux extrémités. L'indentation d'origine est perdue.

L. 164 Ligne vide.

- L. 165** Une ligne qui commence par > est une citation Markdown : elle appartient à un bloc.
- L. 166** `nue.lstrip("> ")` : attention, `lstrip` reçoit un **ensemble de caractères**, pas un préfixe. Il retire au début tous les > et espaces, dans n'importe quel ordre (> > `texte` donne `texte`). `.rstrip()` nettoie la fin. Une ligne composée seulement de > devient une chaîne vide.
- L. 167** `type_bloc is None` : c'est la *première* ligne du bloc.
- L. 168–169** Le type du bloc est fixé une fois pour toutes par sa première ligne : `RE_AVERTISSEMENT.search` cherche le motif *n'importe où* dans la ligne. S'il est trouvé, c'est un avertissement, sinon une note.
- L. 170** La ligne est ajoutée au bloc.
- L. 171** `continue` : on passe directement à la ligne suivante, sans exécuter la suite de la boucle.

```

172
173     if bloc:
174         cloturer()
175
176     if nue == "---":
177         continue
178     if nue.startswith("**Source officielle"):
179         continue
180     if nue.startswith("(La suite de l'article"):
181         continue
182
183     normatif.append(nue)
184
185 cloturer()
186 texte = re.sub(r"\n{3,}", "\n\n", "\n".join(normatif)).strip()
187 return texte, avertissements, notes

```

- L. 172** Ligne vide.
- L. 173–174** La ligne n'est pas une citation mais un bloc était ouvert : le bloc est terminé, on le clôture. Conséquence : une ligne vide sans > entre deux paragraphes d'une citation coupe celle-ci en deux blocs, et le second prend le type de sa propre première ligne.
- L. 175** Ligne vide.
- L. 176–177** `---` (ligne horizontale Markdown) : ignorée.
- L. 178–179** La ligne « `**Source officielle**` » (métadonnée éditoriale) : ignorée.
- L. 180–181** Marqueur « `*(La suite de l'article...*` » : ignoré.
- L. 182** Ligne vide.
- L. 183** Tout le reste est du texte normatif. Les lignes vides (`""`) sont aussi ajoutées : ce sont elles qui gardent la séparation entre paragraphes, utilisée plus tard par `sous_decouper`.
- L. 184** Ligne vide.
- L. 185** Dernier appel à `cloturer` : si la section se termine par un bloc >, aucune ligne normale n'a déclenché sa clôture.
- L. 186** On recolle les lignes normatives, puis `re.sub(r"\n{3,}", "\n\n", ...)` remplace toute suite de trois retours à la ligne ou plus par deux : au plus une ligne vide entre paragraphes. `.strip()` retire les lignes vides au début et à la fin.
- L. 187** Renvoi du tuple (texte, avertissements, notes).

2.6 Tableaux et articles trop longs (lignes 188 à 269)

```
188
189
190 # =====
191 # SOUS-DECOUPAGE DES ARTICLES TROP LONGS
192 # =====
193
194 def eclater_tableau(texte: str) -> list[tuple[str, str]] | None:
195     """
196     Transforme un tableau markdown en une liste de (etiquette, phrase).
197
198     POURQUOI : le tableau de l'article premier fait 2462 caracteres et
199     melange 10 especes. Le signal "sanglier" s'y noyait parmi lievre,
200     becasse et tourterelle : un vecteur unique ne peut pas représenter
201     10 regimes de dates differents.
202
203     Meme principe que l'article 12 : un tableau est une base de donnees
204     deguisee en prose, pas de la prose. On le decoupe par LIGNE.
205
206     Retourne None si le texte n'est pas un tableau exploitable.
207     """
208     lignes = [l for l in texte.splitlines() if RE_LIGNE_TABLEAU.match(l)]
209     if len(lignes) < 3:
210         return None
```

eclater_tableau : une ligne de tableau = un chunk

L. 188–189 Lignes vides.

L. 190–192 Bannière « SOUS-DECOUPAGE DES ARTICLES TROP LONGS ».

L. 193 Ligne vide.

L. 194 Définition : renvoie une liste de couples (etiquette, phrase), ou None.

L. 195–207 Docstring. Le tableau de l'article premier fait 2462 caractères et mélange 10 espèces. Or un embedding est **un seul vecteur de taille fixe** : il ne peut pas représenter dix régimes de dates différents, et le signal « sanglier » s'y noie. Un tableau est en réalité une base de données écrite sous forme de texte : on le découpe *par ligne*.

L. 208 Compréhension de liste : on ne garde que les lignes qui commencent par |.

L. 209–210 Moins de 3 lignes (il faut au minimum entête, séparateur et une ligne de données) : ce n'est pas un tableau exploitable.

```
211
212     entetes, donnees = None, []
213     for l in lignes:
214         if RE_LIGNE_SEPARATRICE.match(l):
215             continue
216         cells = [c.strip() for c in l.strip().strip("|").split("|")]
217         if entetes is None:
218             entetes = cells
219         else:
220             donnees.append(cells)
221
```

```

222     if not entetes or not donnees:
223         return None

```

L. 211 Ligne vide.

L. 212 `entetes` vaut `None` tant que la ligne d'entête n'est pas lue ; `donnees` recevra les lignes suivantes.

L. 213 Parcours des lignes du tableau.

L. 214–215 La ligne `|---|---` est ignorée.

L. 216 Découpage d'une ligne en cellules, en trois étapes : `l.strip()` retire les espaces, `.strip("|")` retire les barres du début et de la fin, `.split("|")` coupe aux barres restantes ; chaque cellule est ensuite nettoyée par `c.strip()`.

Exemple :

```

| Sanglier | 5 octobre 2025 | 1er février 2026 |
donne
["Sanglier", "5 octobre 2025", "1er février 2026"]
.

```

L. 217–218 La première ligne non séparatrice est l'entête.

L. 219–220 Les suivantes sont des lignes de données.

L. 221 Ligne vide.

L. 222–223 Sans entête ou sans données : `None`.

```

224
225     sortie = []
226     for cells in donnees:
227         if len(cells) != len(entetes):
228             continue
229         # Première colonne = sujet de la ligne (l'espece)
230         sujet = cells[0]
231         details = ", ".join(f"{e} : {c}" for e, c in zip(entetes[1:], cells[1:]))
232         sortie.append((sujet, f"{sujet}. {details}."))
233     return sortie or None

```

L. 224 Ligne vide.

L. 225 Liste résultat.

L. 226 Parcours des lignes de données.

L. 227–228 Si le nombre de cellules diffère de celui de l'entête (ligne mal formée), la ligne est **ignorée sans message**.

L. 229 Commentaire : la première colonne est le sujet de la ligne.

L. 230 `sujet` = première cellule (l'espece).

L. 231 `zip(entetes[1:], cells[1:])` associe chaque entête (sauf la première) à la cellule correspondante. Pour chaque couple, on écrit "**entête** : **valeur**", puis on joint le tout avec des virgules.

Résultat :

```

Date d'ouverture : 5 octobre 2025, Date de fermeture : 1er février 2026
.

```

L. 232 On ajoute le couple (sujet, phrase) ; la phrase vaut "sujet. détails."

Attention : si le sujet finit déjà par un point, on obtient deux points : « (El khedra).. Date d'ouverture », visible dans `chunks.json`.

L. 233 `sortie` or `None` : une liste vide est « fausse », donc on renvoie `None` si aucune ligne n'a été retenue.

contient_tableau

```
234
235
236 def contient_tableau(texte: str) -> bool:
237     """Un tableau markdown ne doit JAMAIS etre coupe au milieu."""
238     return any(RE_LIGNE_TABLEAU.match(l) for l in texte.splitlines())
```

L. 234–235 Lignes vides.

L. 236 Définition : renvoie un booléen.

L. 237 Docstring : un tableau Markdown ne doit jamais être coupé au milieu.

L. 238 `any(...)` renvoie `True` dès qu'un élément est vrai, et s'arrête là (évaluation *court-circuit*). Une seule ligne commençant par `|` suffit.

sous_decouper : couper un article long par paragraphes

```
239
240
241 def sous_decouper(texte: str) -> list[str]:
242     """
243     Regroupe les paragraphes en blocs de <= TAILLE_MAX_CHUNK caracteres.
244
245     Pourquoi : un embedding est un vecteur de taille FIXE. Plus le texte est
246     long, plus le sens s'y dilue. L'article 3 de l'arrete melange redevances,
247     eperviers, faucons, slougui, sanglier et etourneaux : un seul vecteur ne
248     peut pas represente tout ca.
249
250     On ne coupe jamais au milieu d'un paragraphe. Et on laisse les tableaux
251     entiers meme s'ils dépassent : le tableau de l'article premier fait
252     ~2500 caracteres mais forme un bloc coherent.
253     """
254     if contient_tableau(texte) or len(texte) <= config.TAILLE_MAX_CHUNK:
255         return [texte]
256
257     paragraphes = [p.strip() for p in re.split(r"\n\s*\n", texte) if p.strip()]
258     blocs, courant = [], []
```

L. 239–240 Lignes vides.

L. 241 Définition : renvoie une liste de blocs de texte.

L. 242–253 Docstring. Un embedding est un vecteur de **taille fixe** (1024 nombres) : plus le texte est long, plus son sens est dilué. L'article 3 de l'arrêté mélange redevances, éperviers, faucons,

slougui, sanglier et étourneaux : un seul vecteur ne peut pas bien représenter tout cela. On ne coupe jamais au milieu d'un paragraphe, et on laisse les tableaux entiers.

L. 254 Si le texte contient un tableau, ou s'il est assez court ($\leq \text{TAILLE_MAX_CHUNK} = 1500$ caractères)...

L. 255 ... on le renvoie tel quel, dans une liste à un seul élément. Renvoyer toujours une liste simplifie le code appelant.

L. 256 Ligne vide.

L. 257 `re.split(r"\n\s*\n", texte)` découpe aux lignes vides (qui peuvent contenir des espaces) : on obtient les paragraphes. La compréhension les nettoie et élimine les vides.

L. 258 `blocs = résultat ; courant = paragraphes du bloc en construction.`

```
259
260     for p in paragraphes:
261         if courant and sum(len(x) for x in courant) + len(p) > config.TAILLE_MAX_CHUNK:
262             blocs.append("\n\n".join(courant))
263             courant = [p]
264         else:
265             courant.append(p)
266
267     if courant:
268         blocs.append("\n\n".join(courant))
269     return blocs
```

L. 259 Ligne vide.

L. 260 Parcours des paragraphes.

L. 261 Condition de fermeture du bloc courant :

- `courant` n'est pas vide : sans cette condition, un paragraphe seul de plus de 1500 caractères produirait un bloc vide ; avec elle, il est accepté seul dans son bloc ;
- **et** la taille du bloc plus celle du nouveau paragraphe dépasserait 1500.

La somme ne compte pas les "`\n\n`" ajoutés au recollage : un bloc peut dépasser 1500 de quelques caractères. La somme est recalculée à chaque tour : c'est un coût quadratique, négligeable ici.

L. 262 Le bloc courant est fermé : paragraphes joints par une ligne vide.

L. 263 Le nouveau paragraphe ouvre le bloc suivant.

L. 264–265 Sinon, le paragraphe s'ajoute au bloc courant.

L. 266 Ligne vide.

L. 267–268 Le dernier bloc en construction est ajouté.

L. 269 Renvoi des blocs.

2.7 Préfixes contextuels (lignes 270 à 327)

```
270
271
```

```

272 # =====
273 # PREFIXE CONTEXTUEL
274 # =====
275
276 def _libelle_court(meta: dict) -> str:
277     """
278     Version abregee de la source, pour le prefixe d'embedding.
279     "Arrêté du 27 août 2025 (JORT n° 107 du 29 août 2025)" -> "Arrêté chasse 2025/2026"
280     """
281     sid = meta.get("source_id", "")
282     return {
283         "arrete_2025": "Arrêté chasse 2025/2026",
284         "code_forestier": "Code forestier",
285         "loi_69_33": "Loi armes 69-33",
286         "code_forestier_camper": "Code forestier (camping)",
287         "arretes_parcs_1984": "Arrêtés parcs nationaux 1984",
288         "production_bio_apiculture": "Apiculture bio (arrêté 2005)",
289         "preparation_bio_miel": "Préparation miel bio (arrêté 2005)",
290         "guide_technique_apiculture": "Guide technique apiculture",
291     }.get(sid, meta["source"][:40])

```

Un article isolé (« Il est interdit de... ») ne dit pas de quelle loi il vient. On lui ajoute donc un **préfixe**. Le code en construit **deux versions**, pour deux usages différents.

`_libelle_court`

L. 270–271 Lignes vides.

L. 272–274 Bannière « PREFIXE CONTEXTUEL ».

L. 275 Ligne vide.

L. 276 Le `_` initial est une convention Python : fonction *interne* au module, à ne pas importer ailleurs.

L. 277–280 Docstring : la source longue (« Arrêté du 27 août 2025 (JORT n° 107...) ») devient « Arrêté chasse 2025/2026 ».

L. 281 `meta.get("source_id", "")` : lecture sans risque de `KeyError` ; renvoie "" si la clé est absente.

L. 282 Début d'un dictionnaire littéral, créé à chaque appel.

L. 283–290 Table de correspondance `source_id` → libellé court, une entrée par document du corpus.

L. 291 `.get(sid, meta["source"][:40])` : on cherche le libellé ; s'il est absent, on prend les 40 premiers caractères de la source. Pour tout nouveau document, il faut penser à ajouter une entrée ici, sinon le libellé est tronqué.

construire_prefixe : le préfixe LONG

```

292
293
294 def construire_prefixe(meta: dict, parents: list[str], titre: str) -> str:
295     """
296     PREFIXE LONG - pour BM25 uniquement, et pour l'affichage.
297

```

```

298     Il porte la source complete et toute la hierarchie. Sur un index lexical
299     c'est gratuit : BM25 pondere par l'IDF, donc les mots repetes dans les
300     160 chunks ("arrêté", "titre") ont un poids quasi nul automatiquement.
301     """
302     morceaux = [meta["source"]]
303     for p in parents:
304         if p.startswith(("Titre", "Chapitre", "Partie", "Section")):
305             morceaux.append(p)
306     morceaux.append(titre)
307     return " - ".join(morceaux)

```

L. 292–293 Lignes vides.

L. 294 Définition : reçoit les métadonnées, la pile des parents et le titre.

L. 295–301 Docstring. Le préfixe long sert à BM25 et à l’affichage. Sur un index lexical, il ne coûte rien : BM25 pondère chaque mot par son **IDF** (*Inverse Document Frequency*). Un mot présent dans presque tous les chunks (« arrêté », « titre ») a un poids quasi nul.

L. 302 La liste commence par la source complète.

L. 303 Parcours des parents, du plus général au plus précis.

L. 304 `str.startswith` accepte un *tuple* de préfixes : vrai si l’un d’eux correspond. On ne garde que les parents *structurels* (Titre, Chapitre, Partie, Section). Sont exclus le titre H1 du document et un parent « Article 12 ».

L. 305 Ajout du parent retenu.

L. 306 Ajout du titre de la section.

L. 307 Assemblage avec « — ».

Exemple : « Arrêté du 27 août 2025 (JORT...) — Titre premier : Réglementation générale — Article premier ».

construire_prefixe_court : le préfixe COURT

```

308
309
310 def construire_prefixe_court(meta: dict, titre: str, extra: str = "") -> str:
311     """
312     PREFIXE COURT - pour l'EMBEDDING.
313
314     POURQUOI DEUX PREFIXES :
315     un embedding est un vecteur de taille fixe ou CHAQUE token compte. Le
316     prefixe long fait ~25 tokens IDENTIQUES sur les 57 chunks de l'arrete.
317     Sur un chunk court, il representait la moitie du contenu : tous les
318     vecteurs se ressemblaient et la partie discriminante pesait trop peu.
319     C'est ce qui faisait remonter l'article 10 (colportage) en tete de trois
320     questions sans rapport.
321
322     Contrairement a BM25, l'embedding n'a AUCUN mecanisme equivalent a l'IDF
323     pour ignorer ce qui est commun a tout le corpus. Il faut donc le lui
324     epargner : ici ~6 tokens au lieu de 25.
325     """
326     base = f"{_libelle_court(meta)}, {titre}"
327     return f"{base}. {extra}" if extra else base

```

L. 308–309 Lignes vides.

L. 310 Définition ; **extra** est facultatif (chaîne vide par défaut).

L. 311–325 Docstring, la justification la plus importante du fichier. Dans un embedding, **chaque token compte**. Le préfixe long ajoutait environ 25 tokens identiques aux 57 chunks de l'arrêté. Sur un chunk court, cela représentait la moitié du contenu : tous les vecteurs se ressemblaient, et c'est ce qui faisait remonter l'article 10 (colportage) pour trois questions sans rapport. Contrairement à BM25, l'embedding **n'a aucun mécanisme équivalent à l'IDF** pour ignorer ce qui est commun à tout le corpus : il faut donc le lui épargner (environ 6 tokens au lieu de 25).

L. 326 **base** = « libellé court, titre ».

L. 327 Si **extra** n'est pas vide, on l'ajoute après un point (le nom du gouvernorat, l'espèce d'une ligne de tableau...).

	Préfixe long	Préfixe court
Utilisé par	BM25 (texte_index) et affichage	embedding (texte_embedding)
Contenu	source complète + Titre/Chapitre + titre	libellé court + titre (+ extra)
Pourquoi	l'IDF neutralise les mots répétés	aucun IDF : les mots répétés diluent le vecteur

2.8 `charger_manifeste` (lignes 328 à 345)

```
328
329
330 # =====
331 # CHARGEMENT DES METADONNEES D'UN DOMAINE
332 # =====
333
334 def charger_manifeste(dossier_domaine: Path) -> dict:
335     """
336     Lit data/corpus/<domaine>/_manifest.json s'il existe.
337
338     Le nom du dossier donne le DOMAINE, mais pas le rang normatif ni la
339     validite : c'est le role du manifeste. S'il est absent (cas de camping/
340     ou app/), on applique META_DEFAULT et tout fonctionne quand meme.
341     """
342     f = dossier_domaine / "_manifest.json"
343     if f.exists():
344         return json.loads(f.read_text(encoding="utf-8"))
345     return {}
```

L. 328–329 Lignes vides.

L. 330–332 Bannière « CHARGEMENT DES METADONNEES D'UN DOMAINE ».

L. 333 Ligne vide.

L. 334 Définition : reçoit le dossier d'un domaine (`Path`), renvoie un dictionnaire.

L. 335–341 Docstring. Le nom du dossier donne le domaine, mais pas le rang normatif ni la validité : c'est le rôle du manifeste. S'il est absent, `META_DEFAULT` s'applique.

L. 342 L'opérateur `/` est redéfini par `Path` pour *joindre des chemins* : `data/corpus/chasse / "_manifest.json"`.

L. 343 Le fichier existe-t-il ?

L. 344 Lecture en UTF-8, puis `json.loads` convertit le texte JSON en dictionnaire Python. Le manifeste est indexé par *nom de fichier* :

```
{"code_forestier_chasse.md": {"source": ..., "rang_normatif": "loi", ...}}
```

. Un JSON mal formé lève une exception et interrompt tout le chunking (pas de `try`).

L. 345 Pas de manifeste : dictionnaire vide.

2.9 chunker_fichier : le cœur du découpage (lignes 346 à 438)

```
346
347
348 # =====
349 # TRAITEMENT D'UN FICHIER
350 # =====
351
352 def chunker_fichier(chemin: Path, domaine: str, meta_fichier: dict) -> list[dict]:
353     texte = chemin.read_text(encoding="utf-8")
354     chunks = []
355
356     for sec in decouper_sections(texte):
357         corps, avertissements, _ = separer_contenu(sec["lignes"])
358         titre = sec["titre"]
359         prefixe = construire_prefixe(meta_fichier, sec["parents"], titre)
```

Pour chaque section, **un seul cas** s'applique : gouvernorat, section non-article, ou article (avec ou sans tableau). Les avertissements sont traités en plus, pour les articles.

```
section
+-- EXCEPTION 1 : titre "Gouvernorat de X" -> 1 chunk "zone_geographique"
+-- EXCEPTION 2 : pas un article (Visas...) -> 1 chunk "contexte" si > 200 caracteres
+-- CAS NORMAL : un article
    +-- EXCEPTION 4 : contient un tableau -> 1 chunk "tableau" par ligne
    |                                     + 1 chunk "norme" (texte hors tableau)
    +-- sinon                           -> 1 ou N chunks "norme" (sous_decouper)
    +-- EXCEPTION 3 (en plus) : chaque avertissement -> 1 chunk "avertissement"
```

L. 346–347 Lignes vides.

L. 348–350 Bannière « TRAITEMENT D'UN FICHIER ».

L. 351 Ligne vide.

L. 352 Définition : chemin du fichier, nom du domaine, métadonnées du fichier. Pas de docstring.

L. 353 Lecture du fichier entier en UTF-8.

L. 354 Liste des chunks produits pour ce fichier.

L. 355 Ligne vide.

L. 356 Boucle sur les sections produites par `decouper_sections`.

L. 357 Déballage du tuple renvoyé par `separer_contenu`. Le troisième élément (les notes) est affecté à `_` : convention Python pour « valeur volontairement ignorée ». Les notes de fidélité disparaissent donc ici.

L. 358 Raccourci vers le titre de la section.

L. 359 Préfixe long calculé pour toutes les sections, même celles qui seront ignorées.

```
360
361     # ---- EXCEPTION 1 : sous-sections géographiques de l'article 12 ----
362     m_gouv = RE_GOUVERNORAT.match(titre)
363     if m_gouv and corps:
364         # "### Gouvernorat de Siliana" n'est pas un article : c'est une
365         # subdivision de l'article 12. On remonte la pile des parents
366         # pour retrouver l'article de rattachement.
367         art_parent = next((numero_article(p) for p in reversed(sec["parents"])
368                           if numero_article(p)), None)
369         gouv = m_gouv.group(1).strip()
370         court = construire_prefixe_court(
371             meta_fichier, f"Article {art_parent}",
372             f"Réserves de chasse interdites, gouvernorat de {gouv}")
373         chunks.append(_fabriquer(meta_fichier, domaine, art_parent,
374                                 slug(gouv), titre, prefixe, court, corps,
375                                 "zone_geographique", gouv))
376     continue
```

L. 360 Ligne vide.

L. 361 Commentaire : exception 1, les sous-sections géographiques de l'article 12.

L. 362 Le titre est-il « Gouvernorat de X » ?

L. 363 Oui, et la section a du contenu : on la traite.

L. 364–366 Commentaires : « ### Gouvernorat de Siliana » n'est pas un article mais une subdivision de l'article 12, qu'il faut retrouver dans la pile des parents.

L. 367–368 `next(générateur, None)` renvoie le premier élément produit par le générateur, ou `None` s'il n'en produit aucun. Le générateur parcourt les parents **du plus proche au plus lointain** (`reversed`) et ne produit que ceux qui sont des articles. Résultat : `art_parent = "12"`. (`numero_article` est appelé deux fois par parent : petite redondance.)

L. 369 Nom du gouvernorat : groupe 1 de la regex, sans espaces.

L. 370–372 Préfixe court avec titre « Article 12 » et extra « Réserves de chasse interdites, gouvernorat de Siliana ». Ce texte est écrit en dur : toute section « Gouvernorat de... » de n'importe quel domaine recevrait ce libellé.

L. 373–375 Création du chunk par `_fabriquer`. Arguments, dans l'ordre : métadonnées, domaine, article ("12"), sous-clé `slug(gouv)`, titre, préfixe long, préfixe court, texte (`corps`), type "zone_geographique", gouvernorat. Identifiant obtenu : `arrete_2025_art12_siliana` (24 chunks de ce type dans le corpus).

L. 376 `continue` : section traitée, on passe à la suivante.

```
377
378     article = numero_article(titre)
379
```

```

380     # ---- EXCEPTION 2 : sections qui ne sont pas des articles -----
381     if article is None:
382         # Visas, Preamble, Lacunes connues. Non normatifs mais utiles.
383         # "Lacunes connues" permet au bot de répondre "cette information
384         # n'est pas dans mon corpus" au lieu d'inventer.
385         if corps and len(corps) > config.TAILLE_MINALERTE:
386             court = construire_prefixe_court(meta_fichier, titre)
387             chunks.append(_fabriquer(meta_fichier, domaine, None,
388                                     slug(titre), titre, prefixe, court,
389                                     corps, "contexte", None))
390             continue

```

L. 377 Ligne vide.

L. 378 Numéro d'article du titre, ou None.

L. 379 Ligne vide.

L. 380 Commentaire : exception 2, sections qui ne sont pas des articles.

L. 381 Ce n'est pas un article...

L. 382–384 Commentaires : Visas, Préambule, « Lacunes connues ». Ces sections ne sont pas normatives mais utiles : « Lacunes connues » permet au bot de répondre « cette information n'est pas dans mon corpus » au lieu d'inventer.

L. 385 ... on ne crée un chunk que si le texte dépasse TAILLE_MINALERTE (200 caractères). Cela élimine les titres vides (« ## Chapitre II » sans texte propre).

L. 386 Préfixe court avec le titre de la section.

L. 387–389 Chunk de type "contexte", sans article. La sous-clé est slug(titre). Identifiant réel : arrete_2025_artvisas_visas.

L. 390 continue : une section trop courte est ignorée sans message.

Attention : c'est aussi le cas des documents sans articles (app/, guides d'apiculture) : chacune de leurs sections est un « contexte ». Vérifié sur aide_application.md : trois réponses de la FAQ font moins de 200 caractères et **ne sont jamais indexées** : « Puis-je créer une alerte sans réseau ? » (148), « Comment suivre mes signalements ? » (173), « Les réponses du chatbot ont-elles une valeur légale ? » (160). Le chatbot ne peut donc pas répondre à ces questions.

```

391
392     # ---- CAS NORMAL : un article -----
393     if corps:
394         # --- EXCEPTION 4 : article contenant un tableau -----
395         # Une ligne de tableau = un chunk. Sinon les 10 especes du
396         # tableau de l'article premier se noient dans un seul vecteur.
397         lignes_tab = eclater_tableau(corps) if contient_tableau(corps) else None

```

L. 391 Ligne vide.

L. 392 Commentaire : cas normal, un article.

L. 393 L'article a-t-il du texte ? Un article sans corps ne produit pas de chunk « norme », mais ses avertissements sont traités quand même (ligne 429, hors de ce if).

L. 394–396 Commentaires : exception 4, une ligne de tableau = un chunk.

L. 397 Expression conditionnelle : on n'appelle `eclater_tableau` que si le texte contient un tableau ; sinon `lignes_tab = None`.

```
398
399         if lignes_tab:
400             # le texte hors tableau (alineas qui suivent) devient un chunk
401             hors_tableau = "\n".join(
402                 l for l in corps.splitlines()
403                 if not RE_LIGNE_TABLEAU.match(l)).strip()
404
405             for sujet, phrase in lignes_tab:
406                 court = construire_prefixe_court(meta_fichier, titre, sujet)
407                 chunks.append(_fabriquer(
408                     meta_fichier, domaine, article, slug(sujet, 30), titre,
409                     f"{prefixe} - {sujet}", court, phrase, "tableau", None))
410
411             if len(hors_tableau) > config.TAILLE_MINALERTE:
412                 court = construire_prefixe_court(meta_fichier, titre)
413                 chunks.append(_fabriquer(
414                     meta_fichier, domaine, article, "suite", titre,
415                     prefixe, court, hors_tableau, "norme", None))
```

L. 398 Ligne vide.

L. 399 Le tableau est exploitable (liste non vide).

L. 400 Commentaire : le texte hors tableau devient un chunk.

L. 401–403 `hors_tableau` : on garde les lignes du corps qui *ne* sont *pas* des lignes de tableau (expression génératrice passée à `join`), puis on nettoie.

L. 404 Ligne vide.

L. 405 Pour chaque ligne du tableau : déballage du couple (`sujet`, `phrase`).

L. 406 Préfixe court avec `extra = sujet` : l'espèce apparaît dans le préfixe *et* dans la phrase, ce qui renforce son poids dans le vecteur.

L. 407–409 Chunk de type "tableau" : sous-clé `slug(sujet, 30)` (tronquée à 30 caractères), préfixe long complété par « — sujet », texte = la phrase. Identifiant réel :

`arrete_2025_art1_lievre_perdrix_caille_sedentai`

. Deux lignes dont les 30 premiers caractères seraient identiques produiraient le même identifiant.

L. 410 Ligne vide.

L. 411 Le texte hors tableau fait plus de 200 caractères...

L. 412 ... préfixe court sans `extra`...

L. 413–415 ... et chunk "norme" de sous-clé "suite" (`arrete_2025_art1_suite`). En dessous de 200 caractères, ce texte est perdu.

```
416         else:
417             blocs = sous_decouper(corps)
418             for i, bloc in enumerate(blocs, start=1):
419                 # Si l'article a été sous-découpe, on l'indique au LLM
420                 # pour qu'il sache qu'il ne voit qu'un extrait.
```

```

421         suffixe = f" (partie {i}/{len(blocs)})" if len(blocs) > 1 else ""
422         court = construire_prefixe_court(meta_fichier, titre)
423         chunks.append(_fabriquer(meta_fichier, domaine, article,
424                                 f"p{i}" if len(blocs) > 1 else None,
425                                 titre, prefixe + suffixe, court,
426                                 bloc, "norme", None))

```

L. 416 Pas de tableau, ou tableau mal formé (`eclater_tableau` a renvoyé `None`). Dans ce dernier cas, `sous_decouper` renvoie le texte entier sans le couper.

L. 417 Découpage éventuel en blocs de 1500 caractères au maximum.

L. 418 `enumerate(blocs, start=1)` fournit l'indice `i` (à partir de 1) et le bloc.

L. 419–420 Commentaires : si l'article a été découpé, on l'indique au LLM.

L. 421 `suffixe` = « (partie 2/3) » s'il y a plusieurs blocs, chaîne vide sinon.

L. 422 Préfixe court, identique pour toutes les parties.

L. 423–426 Chunk `"norme"` : sous-clé `"p1"`, `"p2"`... s'il y a plusieurs blocs, sinon `None`. Le suffixe est ajouté au préfixe long. Identifiants réels : `code_forestier_art165` (un bloc), `arrete_2025_art3_p1` (plusieurs).

```

427
428     # ---- EXCEPTION 3 : avertissements de concordance -----
429     for j, avert in enumerate(avertissements, start=1):
430         court = construire_prefixe_court(
431             meta_fichier, titre, "avertissement de concordance")
432         chunks.append(_fabriquer(meta_fichier, domaine, article,
433                                 f"avert{j}",
434                                 f"{titre} - avertissement de concordance",
435                                 prefixe + " - avertissement de concordance",
436                                 court, avert, "avertissement", None))
437
438     return chunks

```

L. 427 Ligne vide.

L. 428 Commentaire : exception 3, les avertissements de concordance.

L. 429 Parcours des avertissements, numérotés à partir de 1. Cette boucle est au niveau de `if corps` : elle s'exécute pour tout article, même sans corps.

L. 430–431 Préfixe court avec l'extra « avertissement de concordance ».

L. 432–436 Chunk `"avertissement"` : sous-clé `avert1`, `avert2`... ; le titre et le préfixe long sont complétés par « — avertissement de concordance ». Identifiants réels : `code_forestier_art161_avert1` et `loi_69_33_art16_avert1`. Le type `"avertissement"` fera ajouter l'étiquette [AVERTISSEMENT DE CONCORDANCE] dans le prompt (`generator.py`).

L. 437 Ligne vide.

L. 438 Renvoi des chunks du fichier.

Attention : un avertissement placé dans une section « Gouvernorat » ou non-article est perdu, car les `continue` des lignes 376 et 390 sautent cette boucle. Aucun cas actuel dans le corpus.

2.10 _fabriquer : la forme d'un chunk (lignes 439 à 479)

```
439
440
441 def _fabriquer(meta_fichier, domaine, article, sous_cle, titre, prefixe,
442               prefixe_court, texte, type_chunk, gouvernement) -> dict:
443     """
444     Fabrique un chunk normalise.
445
446     Les metadonnees restent des types simples (str, int, float, bool) :
447     ni liste, ni dict. D'ou les alias joints en une seule chaine. Cela
448     garde le format lisible en JSON et compatible avec n'importe quel
449     magasin de vecteurs.
450     """
451     ident = f"{meta_fichier['source_id']}_art{article or slug(titre, 20)}"
452     if sous_cle:
453         ident += f"_{sous_cle}"
454
```

L. 439–440 Lignes vides.

L. 441–442 Définition sur deux lignes (continuation implicite dans les parenthèses), avec 10 paramètres positionnels et sans annotations de type.

L. 443–450 Docstring. Les métadonnées ne contiennent que des types simples (`str`, `int`, `float`, `bool`), jamais de listes ni de dictionnaires. C'est pourquoi les alias sont joints en une seule chaîne. Le format reste lisible en JSON et compatible avec n'importe quelle base vectorielle (ChromaDB, par exemple, refuse les listes en métadonnées).

L. 451 Identifiant : `source_id + _art + numéro d'article`. Sans `article`, `article or slug(titre, 20)` prend le slug du titre (20 caractères).

L. 452–453 Si une sous-clé existe (`siliana`, `p2`, `suite`, `avert1...`), on l'ajoute avec `+=` (concaténation). Pour une section non-article, le titre apparaît deux fois :

```
aide_application_artcomment_modifier_mon_comment_modifier_mon_profil
.
```

L. 454 Ligne vide.

```
455     return {
456         "id": ident,
457         # texte_embedding -> COURT, ce qu'on vectorise. Chaque token compte.
458         "texte_embedding": f"{prefixe_court}\n{texte}",
459         # texte_index -> LONG, ce que BM25 lit. L'IDF neutralise le commun.
460         "texte_index": f"{prefixe}\n\n{texte}",
461         # texte_affiche -> ce qu'on MONTRE au LLM et a l'utilisateur
462         "texte_affiche": texte,
463         "prefixe": prefixe,
464         "prefixe_court": prefixe_court,
465         "metadata": {
466             "domaine": domaine,
467             "source": meta_fichier["source"],
468             "source_id": meta_fichier["source_id"],
469             "rang_normatif": meta_fichier["rang_normatif"],
470             "rang_poids": meta_fichier["rang_poids"],
471             "validite": meta_fichier["validite"],
472             "article": article or "",
```

```

473         "titre": titre,
474         "type": type_chunk,
475         "gouvernorat": gouvernorat or "",
476         "alias": " | ".join(extraire_alias_sic(texte)),
477         "nb_caracteres": len(texte),
478     },
479 }

```

L. 455 Renvoi d'un dictionnaire littéral.

L. 456 "id" : identifiant unique du chunk. Il relie `chunks.json`, `vecteurs.npy` (même ordre) et `bm25.pkl`.

L. 457 Commentaire.

L. 458 "texte_embedding" = préfixe **court** + retour à la ligne + texte. C'est ce que `indexer.py` transforme en vecteur.

L. 459 Commentaire.

L. 460 "texte_indexé" = préfixe **long** + ligne vide + texte. C'est ce que BM25 lit.

L. 461 Commentaire.

L. 462 "texte_affiche" = le texte seul. C'est ce qu'on envoie au LLM et qu'on montre à l'utilisateur (via `store.py`).

L. 463 "prefixe" : préfixe long, conservé pour le débogage.

L. 464 "prefixe_court" : préfixe court, conservé pour le débogage.

L. 465 Ouverture du sous-dictionnaire "metadata".

L. 466 `domaine` : nom du dossier (`chasse`, `camper...`). Sert au **filtrage par spécialité** dans `store.py` et `retriever.py`.

L. 467 `source` : nom complet du texte, cité par le LLM et affiché dans les sources.

L. 468 `source_id` : identifiant court du texte.

L. 469 `rang_normatif` (`loi`, `arrete`, `documentaire`) : affiché dans l'en-tête de l'extrait, il permet au LLM d'appliquer la hiérarchie des normes (règle 4 du prompt).

L. 470 `rang_poids` : version numérique du rang (1 = loi, 2 = arrêté, 9 = documentaire). Il n'est lu par aucun autre fichier pour l'instant.

L. 471 `validite` (`permanent`, `saison_2025_2026`) : permet au LLM de signaler une période écoulée (règle 3 du prompt).

L. 472 `article` : numéro ou "". On utilise une chaîne vide plutôt que `None` pour garder des types simples.

L. 473 `titre` : titre de la section.

L. 474 `type` : norme, tableau, zone_géographique, contexte ou avertissement. `main.py` n'affiche comme sources que les trois premiers.

L. 475 `gouvernorat` : nom ou "".

L. 476 `alias` : les formes correctes des coquilles, jointes par " | ". `indexer.py` les ajoute au texte indexé par BM25.

L. 477 `nb_caracteres` : longueur du texte affiché, en *caractères* (pas en tokens). Sert au rapport de contrôle.

L. 478 Fermeture de `metadata`.

L. 479 Fermeture du chunk.

2.11 chunker_corpus : le point d'entrée (lignes 480 à 523)

```
480
481
482 # =====
483 # POINT D'ENTREE : SCAN DE TOUS LES DOMAINES
484 # =====
485
486 def chunker_corpus(verbeux: bool = True) -> list[dict]:
487     """
488     Parcourt data/corpus/<domaine>/*.md et produit tous les chunks.
489     Le nom du sous-dossier devient la metadonnee "domaine".
490     """
491     tous = []
```

L. 480–481 Lignes vides.

L. 482–484 Bannière « POINT D'ENTREE : SCAN DE TOUS LES DOMAINES ».

L. 485 Ligne vide.

L. 486 Définition : `verbeux` (affichage de la progression) vaut `True` par défaut. C'est la fonction appelée par `scripts/build_embeddings.py`.

L. 487–490 Docstring : parcourt `data/corpus/<domaine>/*.md` ; le nom du sous-dossier devient la métadonnée `domaine`.

L. 491 `tous` accumulera les chunks de tous les fichiers.

```
492
493     for dossier in sorted(config.CORPUS_DIR.iterdir()):
494         if not dossier.is_dir():
495             continue
496         domaine = dossier.name
497         manifeste = charger_manifeste(dossier)
498         fichiers = sorted(dossier.glob("*.md"))
499
500         if not fichiers:
501             continue
502         if verbeux:
503             print(f"\n  [{domaine}])")
```

L. 492 Ligne vide.

L. 493 `CORPUS_DIR.iterdir()` liste le contenu de `data/corpus/`. L'ordre renvoyé par le système n'est pas garanti : `sorted` impose un ordre alphabétique stable (`apiculture`, `app`, `camper`, `chasse`). C'est essentiel : `vecteurs.npy` et `bm25.pkl` retrouvent les chunks **par position**, et l'empreinte du cache d'embedding dépend de l'ordre.

L. 494–495 On ignore ce qui n'est pas un dossier (un fichier posé à la racine du corpus).

L. 496 Nom du dossier = `domaine`.

L. 497 Lecture du manifeste du domaine.

L. 498 `glob("*.md")` : les fichiers Markdown du dossier, non récursif (les sous-dossiers sont ignorés). `_manifest.json` n'est pas concerné. Triés pour la même raison qu'à la ligne 493.

L. 499 Ligne vide.

L. 500–501 Dossier sans `.md` : on passe au suivant.

L. 502–503 Affichage du nom du domaine entre crochets.

```
504
505     for f in fichiers:
506         meta = {**config.META_DEFAULT, **manifeste.get(f.name, {})}
507         # Si aucun manifeste, on derive un source_id du nom de fichier
508         # pour que les identifiants de chunks restent uniques.
509         if f.name not in manifeste:
510             meta["source_id"] = slug(f.stem, 30)
511             meta["source"] = f.stem.replace("_", " ")
512             c = chunker_fichier(f, domaine, meta)
513             tous.extend(c)
514             if verbeux:
515                 print(f"    {f.name:38s} -> {len(c):3d} chunks")
```

L. 504 Ligne vide.

L. 505 Parcours des fichiers du domaine.

L. 506 `{**A, **B}` *fusionne* deux dictionnaires : on part de `META_DEFAULT`, puis les clés du manifeste pour ce fichier **écrasent** les valeurs par défaut. `manifeste.get(f.name, {})` renvoie `{}` si le fichier n'est pas décrit.

L. 507–508 Commentaires : sans manifeste, on dérive un `source_id` du nom de fichier pour que les identifiants restent uniques.

L. 509 Le fichier n'est pas décrit dans le manifeste...

L. 510 ... `source_id` = slug du nom sans extension (`f.stem`), limité à 30 caractères. Sans cela, tous ces fichiers auraient `source_id` = "interne" et leurs identifiants pourraient entrer en collision.

L. 511 ... `source` lisible : « aide_application » → « aide application ».

L. 512 Découpage du fichier.

L. 513 `extend` ajoute les *éléments* de la liste `c` à `tous` (`append` ajouterait la liste entière comme un seul élément).

L. 514–515 Affichage : `{f.name:38s}` aligne le nom à gauche sur 38 caractères, `{len(c):3d}` écrit le nombre sur 3 caractères. On obtient une colonne bien alignée.

```
516
517     config.INDEX_DIR.mkdir(parents=True, exist_ok=True)
518     config.CHUNKS_FILE.write_text(
519         json.dumps(tous, ensure_ascii=False, indent=2), encoding="utf-8")
520
521     if verbeux and tous:
522         _rapport(tous)
523     return tous
```

L. 516 Ligne vide.

L. 517 Création de `data/index/` : `parents=True` crée aussi les dossiers intermédiaires, `exist_ok=True` évite une erreur si le dossier existe (équivalent de `mkdir -p`).

L. 518–519 Écriture de `chunks.json`, qui écrase l'ancien fichier. `ensure_ascii=False` garde les accents lisibles (sinon « é » deviendrait `\u00e9`) ; `indent=2` rend le fichier lisible par un humain.

L. 520 Ligne vide.

L. 521–522 Rapport de contrôle seulement en mode verbeux *et* s'il y a des chunks : `_rapport` appelle `min` et divise par `len`, ce qui échouerait sur une liste vide.

L. 523 Renvoi de tous les chunks à l'appelant (`build_embeddings.py`, qui les passe ensuite à `embedder`).

2.12 `_rapport` et exécution directe (lignes 524 à 551)

```
524
525
526 def _rapport(chunks: list[dict]) -> None:
527     """Contrôle qualité affiché après le chunking."""
528     print(f"\n  TOTAL : {len(chunks)} chunks -> {config.CHUNKS_FILE}")
529
530     for cle in ("domaine", "type"):
531         compte = {}
532         for c in chunks:
533             v = c["metadata"][cle]
534             compte[v] = compte.get(v, 0) + 1
535             print(f"  Par {cle} : {compte}")
```

L. 524–525 Lignes vides.

L. 526 Définition : `-> None`, la fonction ne renvoie rien, elle affiche.

L. 527 Docstring : contrôle qualité affiché après le chunking.

L. 528 Nombre total de chunks et chemin du fichier.

L. 529 Ligne vide.

L. 530 Deux passages : un comptage par `domaine`, un par `type`.

L. 531 Dictionnaire de comptage {valeur : nombre}.

L. 532 Parcours de tous les chunks.

L. 533 Valeur de la métadonnée étudiée.

L. 534 Idiomme de comptage : `compte.get(v, 0)` renvoie 0 la première fois, puis on ajoute 1. Équivalent de `collections.Counter`.

L. 535 Affichage, par exemple

```
Par domaine : {'apiculture': 61, 'app': 7, ...}
```

.

```
536
537     tailles = [c["metadata"]["nb_caracteres"] for c in chunks]
538     print(f"  Taille : min={min(tailles)} / moy={sum(tailles)//len(tailles)} "
```

```

539         f"/ max={max(tailles)}")
540
541     courts = [c["id"] for c in chunks
542               if c["metadata"]["nb_caracteres"] < config.TAILLE_MINALERTE
543               and c["metadata"]["type"] == "norme"]
544     if courts:
545         print(f"  Chunks courts (< {config.TAILLE_MINALERTE} car.) : {len(courts)}"
546               f" - sauves par le prefixe contextuel")

```

L. 536 Ligne vide.

L. 537 Liste des longueurs de tous les chunks.

L. 538–539 Minimum, moyenne et maximum. `//` est la division *entière*. Deux f-strings consécutives sont concaténées automatiquement par Python.

L. 540 Ligne vide.

L. 541–543 Identifiants des chunks "norme" de moins de 200 caractères.

L. 544 S'il y en a...

L. 545–546 ... on affiche leur nombre, avec la mention « sauvés par le préfixe contextuel » : un article très court reste compréhensible grâce à la source et au titre ajoutés.

```

547
548
549 if __name__ == "__main__":
550     print("\n=== ETAPE 1 : CHUNKING ===")
551     chunker_corpus()

```

L. 547–548 Lignes vides.

L. 549 `__name__` vaut `"__main__"` seulement quand le fichier est *exécuté* (`python -m rag.chunker`), et non quand il est *importé* par un autre module. Le bloc suivant ne s'exécute donc pas lors d'un `import`.

L. 550 Affichage du titre de l'étape.

L. 551 Lancement du chunking complet, en mode verbeux par défaut. Dernière ligne du fichier.

3 Remarques relevées pendant la lecture

Rien de bloquant pour le fonctionnement actuel, sauf le point 1, qui a un effet réel.

1. **Sections courtes perdues (lignes 385 et 390).** Une section qui n'est pas un article et fait 200 caractères ou moins est ignorée sans message. Dans `aide_application.md`, trois réponses de la FAQ ne sont jamais indexées : « Puis-je créer une alerte sans réseau ? » (148 caractères), « Comment suivre mes signalements ? » (173) et « Les réponses du chatbot ont-elles une valeur légale ? » (160). Piste : un seuil plus bas pour les documents sans articles, ou aucun seuil quand le titre est une question.
2. **RE_NOTE_FIDELITE n'est jamais utilisée (ligne 46).** Tout bloc > qui n'est pas un avertissement est classé « note ».
3. **Double point dans les chunks tableau (ligne 232)** : « (El khedra).. Date d'ouverture ».
4. **Commentaire mal placé (ligne 53)** : il décrit la ligne 52.
5. **Texte avant le premier titre perdu (ligne 129).**
6. **Avertissements perdus hors article (lignes 376 et 390)** : aucun cas actuel.
7. **« Article 1er » non reconnu (ligne 40)** à cause de `\b` : le corpus actuel écrit « Article premier ».
8. **Identifiants redondants (lignes 451–453)** pour les sections non-article, et collision possible si deux lignes de tableau ont les mêmes 30 premiers caractères.
9. **rang_poids n'est lu par aucun autre fichier (ligne 470).**

4 Résumé : le rôle de `chunker.py` dans le RAG

`chunker.py` est la **première étape** du RAG et elle en fixe la qualité : un chunk mal découpé ne peut être rattrapé ni par le retriever, ni par le LLM. Le fichier transforme les `.md` du corpus en environ 320 morceaux autonomes enregistrés dans `data/index/chunks.json`.

- **Découpage sémantique** : un article = un chunk, repéré par la regex « Article N », quel que soit le niveau de titre.
- **Contexte conservé** : une pile de parents garde la hiérarchie Titre → Chapitre → Article.
- **Cas particuliers** : tableaux découpés ligne par ligne, articles longs coupés par paragraphes (1500 caractères au maximum), article 12 découpé par gouvernorat, avertissements de concordance transformés en chunks à part.
- **Nettoyage** : notes éditoriales retirées ; coquilles officielles gardées, avec leur forme correcte en alias pour BM25.
- **Trois textes par chunk** : préfixe court + texte pour l'embedding, préfixe long + texte pour BM25, texte seul pour le LLM.
- **Métadonnées** (domaine, source, rang normatif, validité, article, type) : elles servent à *filtrer* (spécialité de l'utilisateur), *citer* (numéro d'article) et *dater* (validité de la saison) dans les étapes suivantes.

Le fichier suivant dans le flux est `indexer.py` : il transforme ces chunks en vecteurs (`vecteurs.npy`) et en index lexical (`bm25.pkl`).